



Lecture 11

Robustness to Flexible Confounding

Dave Armstrong

Today's Robustness Question

What assumption/abstraction are we perturbing?

- The nuisance functions relating the controls to the treatment and outcome are adequately modeled.

Why might this assumption fail?

- There may be unmodeled nonlinearities and interactions in the nuisance functions relating the controls to the treatment and outcome.

Today's Robustness Question

What assumption/abstraction are we perturbing?

- The nuisance functions relating the controls to the treatment and outcome are adequately modeled.

What kinds of inferential instability can result?

- Estimated treatment effects can be sensitive to how the controls are modeled.
- Conventional parametric models may inadequately remove confounding.

Today's Robustness Question

What assumption/abstraction are we perturbing?

- The nuisance functions relating the controls to the treatment and outcome are adequately modeled.

What tools exist to evaluate robustness?

- Double Machine Learning

Today's Robustness Question

What assumption/abstraction are we perturbing?

- The nuisance functions relating the controls to the treatment and outcome are adequately modeled.

What are the limits of these tools?

- They do not solve omitted variable bias or endogeneity (inherently).

Recovering Treatment Effects

We want to estimate a **treatment effect** τ in the model:

$$Y_i = \tau D_i + g(\mathbf{X}_i) + \varepsilon_i$$

- Y_i — outcome (vote share, conflict onset, support for policy...)
- D_i — treatment or variable of interest (could be continuous)
- $\mathbf{X}_i$ — a vector of **controls** (possibly many)
- $g(\mathbf{X}_i)$ — unknown function; we don't care about its form
- ε_i — idiosyncratic error, $\mathbb{E}[\varepsilon_i | D_i, \mathbf{X}_i] = 0$

Recovering Treatment Effects

We want to estimate a **treatment effect** τ in the model:

$$Y_i = \tau D_i + g(\mathbf{X}_i) + \varepsilon_i$$

- Y_i — outcome (vote share, conflict onset, support for policy...)
- D_i — treatment or variable of interest (could be continuous)
- $\mathbf{X}_i$ — a vector of **controls** (possibly many)
- $g(\mathbf{X}_i)$ — unknown function; we don't care about its form
- ε_i — idiosyncratic error, $\mathbb{E}[\varepsilon_i | D_i, \mathbf{X}_i] = 0$

The Selection Equation

$$D_i = m(\mathbf{X}_i) + v_i$$

where $m(\mathbf{X}_i) = \mathbb{E}[D_i | \mathbf{X}_i]$ and $\mathbb{E}[v_i | \mathbf{X}_i] = 0$.



Why Might Regression Adjustment Fail?

- nonlinear confounding
- interactions among controls
- many controls ($p \rightarrow n$)
- highly collinear controls
- unknown functional form

The Key Theorem

Frisch-Waugh-Lovell (FWL): In OLS, the coefficient on D from

$$Y = \tau D + \mathbf{X}\beta + \varepsilon$$

is **numerically identical** to $\hat{\tau}$ from:

$$\tilde{Y} = \tau \tilde{D} + \varepsilon$$

where $\tilde{Y}$ and $\tilde{D}$ are the **residuals** from regressing Y and D on $\mathbf{X}$.

The Key Theorem

Frisch-Waugh-Lovell (FWL): In OLS, the coefficient on D from

$$Y = \tau D + \mathbf{X}\beta + \varepsilon$$

is **numerically identical** to $\hat{\tau}$ from:

$$\tilde{Y} = \tau \tilde{D} + \varepsilon$$

where $\tilde{Y}$ and $\tilde{D}$ are the **residuals** from regressing Y and D on $\mathbf{X}$.

Intuition: OLS already "partials out" $\mathbf{X}$ from both sides. $\hat{\tau}$ uses only the variation in D *unexplained* by the controls.

The Key Theorem

Frisch-Waugh-Lovell (FWL): In OLS, the coefficient on D from

$$Y = \tau D + \mathbf{X}\beta + \varepsilon$$

is **numerically identical** to $\hat{\tau}$ from:

$$\tilde{Y} = \tau \tilde{D} + \varepsilon$$

where $\tilde{Y}$ and $\tilde{D}$ are the **residuals** from regressing Y and D on $\mathbf{X}$.

Intuition: OLS already "partials out" $\mathbf{X}$ from both sides. $\hat{\tau}$ uses only the variation in D *unexplained* by the controls.

DML generalizes this. Instead of OLS projections, use any ML method to partial out $\mathbf{X}$.

DML in one sentence

Classical regression removes confounding by fitting linear regressions for

$$Y = Xb + \tilde{Y}$$

$$D = Xg + \tilde{D}$$

DML removes confounding by fitting flexible machine learning models for

$$Y = f(X) + \tilde{Y}$$

$$D = g(X) + \tilde{D}$$

Everything else is about making inference valid.

Why Not Just Use OLS with Many Controls?

When the number of controls p is large relative to n :

- OLS is **imprecise** (high variance, may be infeasible if $p > n$)
- **Regularized regression** (LASSO, ridge) introduces **bias**

Why Not Just Use OLS with Many Controls?

When the number of controls p is large relative to n :

- OLS is **imprecise** (high variance, may be infeasible if $p > n$)
- **Regularized regression** (LASSO, ridge) introduces **bias**

Regularization bias problem (Belloni, Chernozhukov, Hansen 2014):

If you run LASSO of Y on $(D, \mathbf{X})$, LASSO may **drop** D or shrink $\hat{\tau}$ toward zero even if $\tau \neq 0$.

Inference after model selection is **invalid** without corrections.

Why Not Just Use OLS with Many Controls?

When the number of controls p is large relative to n :

- OLS is **imprecise** (high variance, may be infeasible if $p > n$)
- **Regularized regression** (LASSO, ridge) introduces **bias**

Regularization bias problem (Belloni, Chernozhukov, Hansen 2014):

If you run LASSO of Y on $(D, \mathbf{X})$, LASSO may **drop** D or shrink $\hat{\tau}$ toward zero even if $\tau \neq 0$.

Inference after model selection is **invalid** without corrections.

Post-double-selection (Belloni et al.) is one fix — but it requires LASSO specifically, and is sensitive to that choice.

The Bias from Naïve Partialing Out

Suppose we run:

1. LASSO of Y on $\mathbf{X} \rightarrow$ residuals $\tilde{Y}$
2. LASSO of D on $\mathbf{X} \rightarrow$ residuals $\tilde{D}$
3. OLS of $\tilde{Y}$ on $\tilde{D} \rightarrow \hat{\tau}$

What goes wrong?

The nuisance estimates $\hat{g}(\mathbf{X}_i)$ and $\hat{m}(\mathbf{X}_i)$ have regularization error.

- This error is as big as ordinary sampling error and thus is not small enough to ignore.

The DML Estimator

Three steps:

Step 1 — Partial out the outcome

Estimate $\hat{g}(\mathbf{X}_i) = \widehat{\mathbb{E}}[Y_i|\mathbf{X}_i]$ using any ML method.

$$\tilde{Y}_i = Y_i - \hat{g}(\mathbf{X}_i)$$

Step 2 — Partial out the treatment

Estimate $\hat{m}(\mathbf{X}_i) = \widehat{\mathbb{E}}[D_i|\mathbf{X}_i]$ using any ML method.

$$\tilde{D}_i = D_i - \hat{m}(\mathbf{X}_i)$$

Step 3 — Recover τ

OLS of $\tilde{Y}$ on $\tilde{D}$:

$$\hat{\tau} = \left(\sum_i \tilde{D}_i^2 \right)^{-1} \sum_i \tilde{D}_i \tilde{Y}_i$$

That's it. The estimator is the FWL residual-on-residual regression, with ML doing the partialing.

Cross-Fitting: The Other Key Ingredient

Problem: If we estimate $\hat{g}$ and $\hat{m}$ on the same sample we use for Step 3, overfitting creates a new bias.

Solution: K -fold cross-fitting

1. Split the sample into K folds (typically $K = 5$)
2. For each fold k :
 - Estimate $\hat{g}^{(-k)}$ and $\hat{m}^{(-k)}$ on the **other** $K - 1$ folds
 - Compute residuals $\tilde{Y}_i$ and $\tilde{D}_i$ for observations in fold k
3. Stack all residuals, run OLS

Cross-Fitting: The Other Key Ingredient

Problem: If we estimate $\hat{g}$ and $\hat{m}$ on the same sample we use for Step 3, overfitting creates a new bias.

Solution: K -fold cross-fitting

1. Split the sample into K folds (typically $K = 5$)
2. For each fold k :
 - Estimate $\hat{g}^{(-k)}$ and $\hat{m}^{(-k)}$ on the **other** $K - 1$ folds
 - Compute residuals $\tilde{Y}_i$ and $\tilde{D}_i$ for observations in fold k
3. Stack all residuals, run OLS

Why it works: Each observation's residual uses a model trained on *different* data. Overfitting does not create systematic bias in the residuals.

Same logic as cross-validation — just applied to the partialing step.

Assumptions

1. Conditional ignorability (the usual one)

$$Y(d) \perp D \mid \mathbf{X} \quad \text{for all } d$$

No unobserved confounders given $\mathbf{X}$. DML cannot partial out variables you didn't measure.

Assumptions

1. Conditional ignorability (the usual one)

$$Y(d) \perp D \mid \mathbf{X} \quad \text{for all } d$$

No unobserved confounders given $\mathbf{X}$. DML cannot partial out variables you didn't measure.

2. The nuisance functions are estimable

The ML models for $g(\mathbf{X})$ and $m(\mathbf{X})$ must converge **faster than** $n^{-1/4}$.

- LASSO achieves this under sparsity
- Random forests and boosting can achieve this under smoothness
- This is a *much* weaker requirement than $\sqrt{n}$ -consistency

Assumptions

1. Conditional ignorability (the usual one)

$$Y(d) \perp D \mid \mathbf{X} \quad \text{for all } d$$

No unobserved confounders given $\mathbf{X}$. DML cannot partial out variables you didn't measure.

2. The nuisance functions are estimable

The ML models for $g(\mathbf{X})$ and $m(\mathbf{X})$ must converge **faster than** $n^{-1/4}$.

- LASSO achieves this under sparsity
- Random forests and boosting can achieve this under smoothness
- This is a *much* weaker requirement than $\sqrt{n}$ -consistency

3. Overlap

$0 < \mathbb{P}(D = 1 | \mathbf{X}) < 1$ for binary D . (Or: $\text{Var}(D | \mathbf{X}) > 0$ for continuous D .)

What DML Does Not Require

You do NOT need:

- Correct specification of $g(\mathbf{X})$
- Correct specification of $m(\mathbf{X})$
- Sparsity (unless using LASSO)
- Knowledge of which controls matter
- Linear functional forms

You DO still need:

- Selection on observables
- Enough data for ML to work
- The residualized treatment–outcome relationship must have the correct functional form.

Machine Learning Models

There are a number of models you could use as learners in the DML framework. We will discuss *Random Forests* and *MARS*.

- Easy to use.
- Not too many tuning parameters.
- Pretty flexible.
- Data requirements are not too heavy.

Random forests use classification and regression trees to which we turn next.

Classification and Regression Trees (CART)

CART works in a decision-tree framework.

- Considering all independent variables, which dichotomization on one of them explains the most variance.
- Conditional on the previous *split*, which next dichotomization explains the most variance.
- Loss function is the well-known residual sum of squares.
- Continue until some fit is not improved (or the nodes/leaves get too small).

Notation

$$f(X_i) = T(X_i, \Theta) \equiv \sum_{b=1}^B c_b I(X_i \in R_b)$$

- $T()$ is a regression tree, with rules Θ regarding tree depth, stopping rules, etc...
- X_i is the data.
- c_b is the predicted value in each of the B regions.
- $I()$ is an indicator function.
- R_b defines the different regions in the space.

Variance of RF Predictions

In addition to finding non-linearities and interactions (i.e., producing a low bias prediction), RF also attempts to produce a low-variance prediction, too.

$$\text{Var} \left(\frac{1}{M} \sum_{m=1}^M T_m \right) = \rho \sigma^2 + \frac{1 - \rho}{M} \sigma^2$$

where

- σ^2 is the variance of a typical tree,
- ρ is the average pairwise correlation between trees.
- If $\rho = 0$ (independence), then

$$\text{Var} = \frac{\sigma^2}{M}$$

so variance shrinks to zero as M grows.

- If $\rho=1$ (perfect correlation),

$$\text{Var} = \sigma^2$$

and averaging accomplishes nothing.

In practice, $0 < \rho < 1$, so averaging still helps, but the residual variance is bounded below by $\rho \sigma^2$.

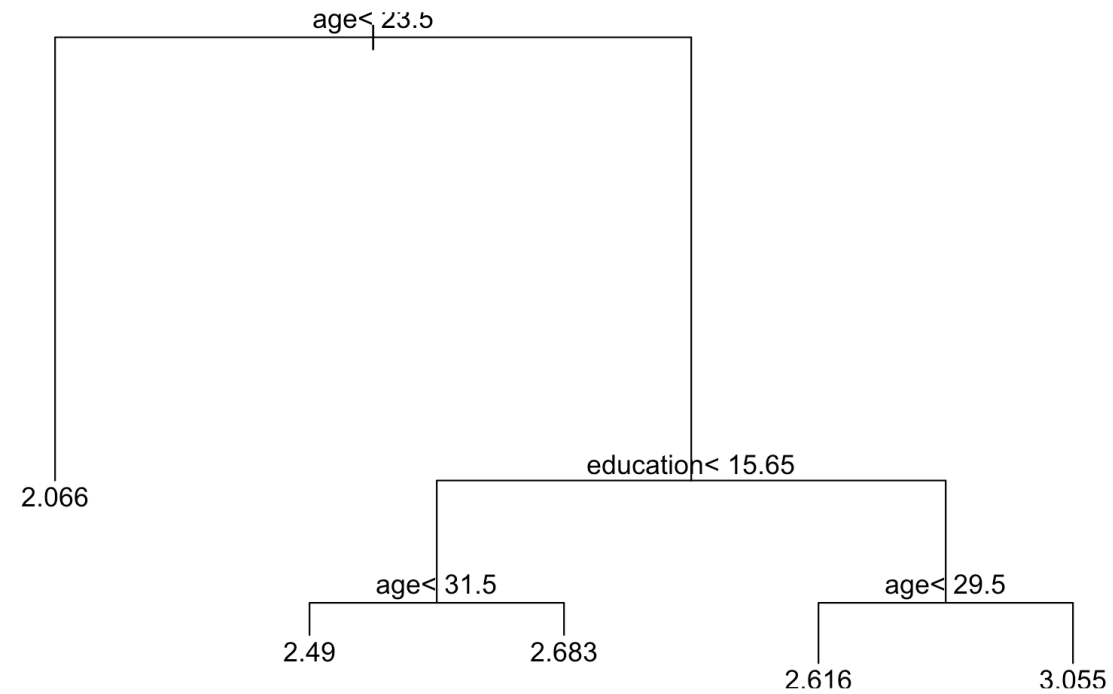


Example

```
library(rpart)
library(dplyr)
library(car)
data(SLID)
SLID$log_wages <- log(SLID$wages)
SLID$male <- as.numeric(SLID$sex == "Male")
SLID <- na.omit(SLID)
mod <- rpart(log(wages) ~ age + education, data=SLID)
mod
```

```
## n= 3987
##
## node), split, n, deviance, yval
##      * denotes terminal node
##
##  1) root 3987 1010.05600 2.619376
##    2) age< 23.5 610    64.40401 2.065618 *
##    3) age>=23.5 3377  724.80820 2.719404
##      6) education< 15.65 2522  478.96440 2.641188
##        12) age< 31.5 551    85.16101 2.490391 *
##        13) age>=31.5 1971  377.77110 2.683344 *
##      7) education>=15.65 855  184.90490 2.950116
##        14) age< 29.5 205    36.60541 2.616058 *
##        15) age>=29.5 650   118.20740 3.055473 *
```

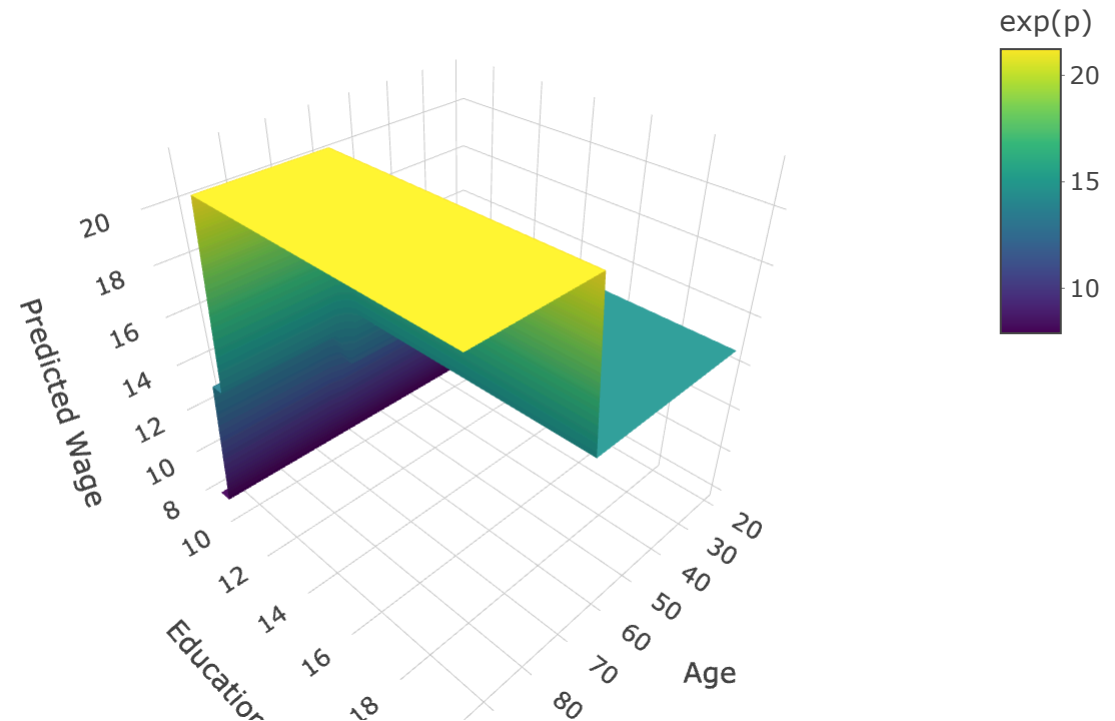
Decision Tree



Surface Plot

```
age.s <- seq(20, 95, length=25)
educ.s <- seq(9, 20, length=25)
cartpred <- function(x,y){
  predict(mod,
    newdata=data.frame(
      age = x,
      education=y))
}
p <- outer(age.s,
  educ.s,
  cartpred)
```

```
library(plotly)
plot_ly() %>%
  add_surface(x=~age.s, y=~educ.s, z=~exp(p))
  layout(
    scene= list(
      xaxis=list(title="Age"),
      yaxis=list(title="Education"),
      zaxis=list(title="Predicted Wage")))
```



(Lack of) Smoothness of CART Models

Problems with CART

- Not particularly smooth - step functions aren't great approximations for smooth curves (though they can do OK).
- No real means for inference here. Bootstrapping can be problematic because the function is "non-regular" (small data changes can result in wild changes in the model)
- In more complicated models, it's difficult to figure out what effects look like.

Ensemble Methods

Ensemble methods produce a bunch of (M) trees to better fit $f(X)$ and to prevent overfitting, with general form:

$$f(X_i) = \sum_{m=1}^M T_m(X_i, \Theta_m)$$

Random Forests

- Draw lots of random samples from the data
- In each random sample, fit a tree on a random subset of the variables
- Average across the trees $\hat{f}_{\text{bag}}(X_i) = \frac{1}{M} \sum_{m=1}^M T_m(X_i, \Theta_m)$

The effectiveness of ensemble averaging depends on the trees making different prediction errors.

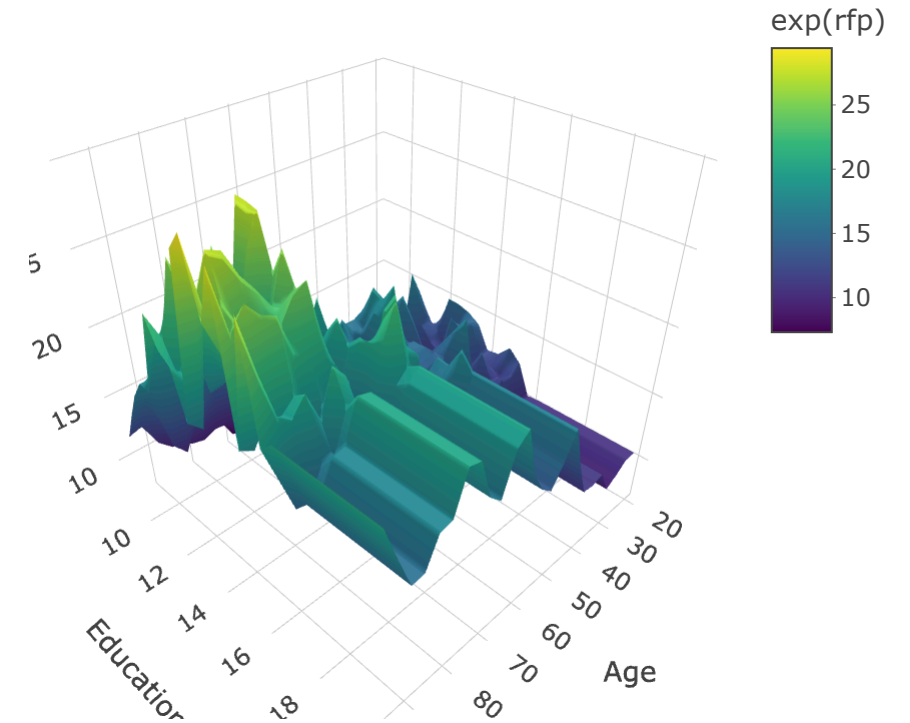
- Highly correlated trees provide little benefit from averaging.
- Random Forests deliberately reduce the correlation among trees by randomly restricting the candidate predictors at each split

Random Forests

CART approximates smooth functions using piecewise-constant regions, producing discontinuous prediction surfaces

- RF is better at approximating smooth functions.

```
library(randomForest)
rfmod <- randomForest(log(wages) ~ age+education,
                      data=SLID,
                      type="regression")
```

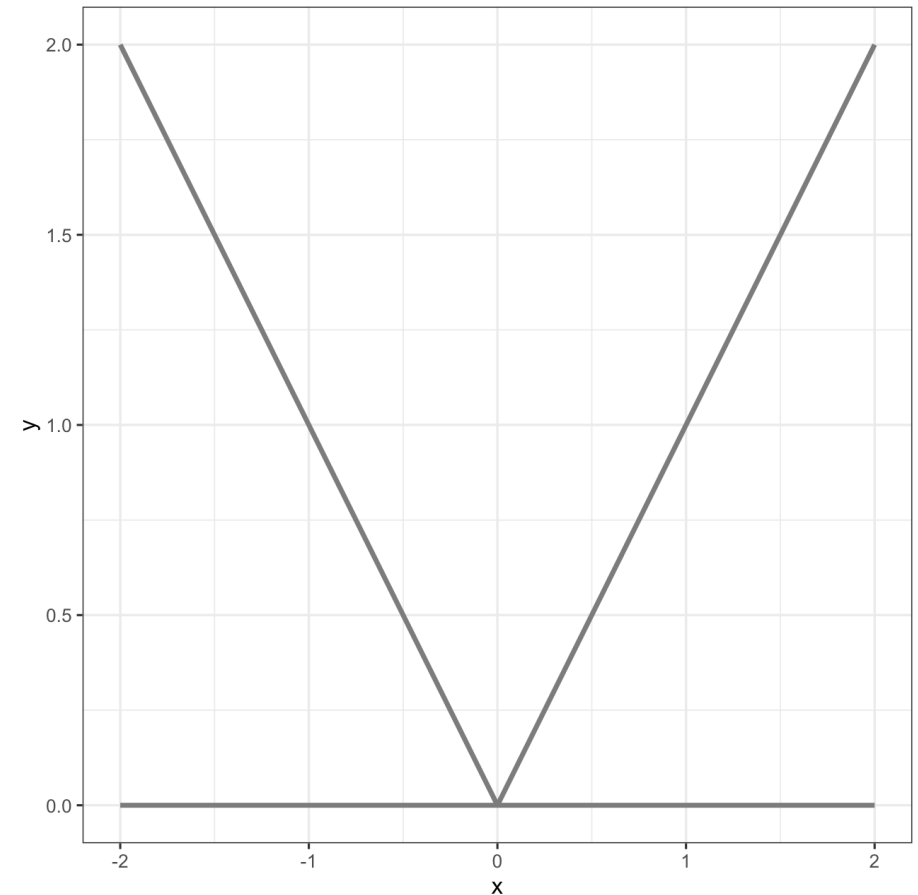


Multivariate Adaptive Regression Splines (MARS)

The main component of MARS is a pair of piecewise linear (hinge) splines.

$$(x - t)_+ = \begin{cases} x - t & \text{if } x > t \\ 0 & \text{otherwise.} \end{cases}$$

$$(t - x)_+ = \begin{cases} t - x & \text{if } x < t \\ 0 & \text{otherwise.} \end{cases}$$



MARS Notation

MARS takes the form:

$$f(x) = \beta_0 + \sum_{m=1}^M \beta_m h_m(x)$$

where h_m is the pair of hinge functions.

Computationally:

- Forward pass - add pairs of hinge functions by reduction in SSRes until all pairs are in.
- Backward pass - take individual functions out by min increase in SSRes until GCV criterion is satisfied.



Interactions

- The `degree` parameter in the R algorithm controls the degree of interaction you want to allow.
- This can make the model really complicated because it's expanding all possible interactions among hinge functions and then pulling them out on the backward pass step.
- This model is more easily constrained (particular w.r.t additivity) than the other models we talked about before.
- You can also identify variables that will enter the model linearly *if they enter the model at all* .



MARS Wages

```
library(earth)
SLID <- SLID %>%
  filter(!is.na(wages) &
         !is.na(education) &
         wages > 0)
emod <- earth(log(wages) ~ age +
              education, data=SLID,
              degree=3)
```

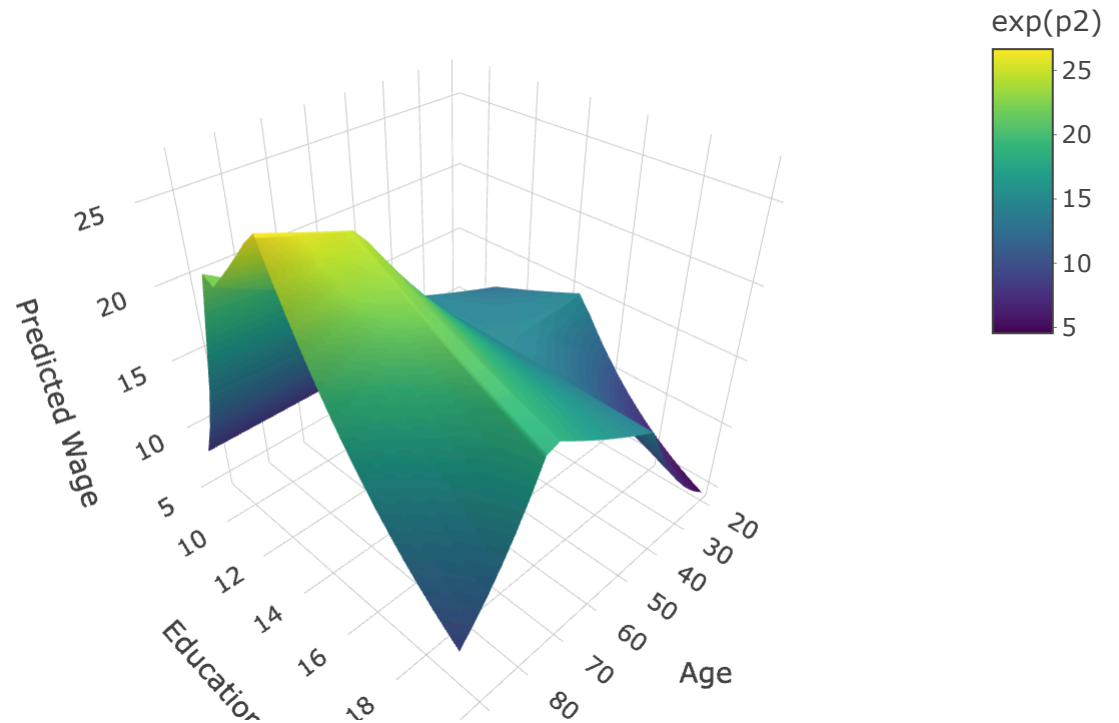
```
summary(emod)
```

```
## Call: earth(formula=log(wages)~age+education, data=SLID, a
##
##                                     coefficients
## (Intercept)                        2.67729521
## h(32-age)                          -0.04911792
## h(age-32)                          -0.03004296
## h(12.6-education)                  -0.15924569
## h(education-12.6)                  0.15407459
## h(age-32) * h(education-17.1)     -0.01138343
## h(age-32) * h(17.1-education)      0.00653049
## h(age-34) * h(education-12.6)      0.00532288
## h(51-age) * h(education-12.6)     -0.00497051
## h(58-age) * h(12.6-education)      0.00417662
## h(age-58) * h(12.6-education)     -0.01522641
##
## Selected 11 of 12 terms, and 2 of 2 predictors
## Termination condition: RSq changed by less than 0.001 at 1
## Importance: age, education
## Number of terms at each degree of interaction: 1 4 6
## GCV 0.1659743   RSS 653.1369   GRSq 0.3451777   RSq 0.3
```

Surface Plot

```
marspred <- function(x,y){  
  predict(emod,  
    newdata=data.frame(age = x,  
                        education=y))  
}  
p2 <- outer(age.s,  
            educ.s,  
            marspred)
```

```
plot_ly() %>%  
  add_surface(x=~age.s, y=~educ.s, z=~exp(p2))  
  layout(  
    scene= list(  
      xaxis=list(title="Age"),  
      yaxis=list(title="Education"),  
      zaxis=list(title="Predicted Wage")  
    ))
```



Choosing the ML Method

The choice of ML for $\hat{g}$ and $\hat{m}$ does not affect the asymptotic distribution of $\hat{\tau}$ — as long as convergence is fast enough.

In practice, choose based on what you think fits the data generating process:

If you believe...	Consider...
True model is sparse (few controls matter)	LASSO, elastic net
Many controls matter, nonlinear interactions	Random forest, MARS
Smooth effects, moderate p	Ridge regression, GAM
No strong prior	Try multiple, compare residuals

Practical tip from Ratkovic: Check that $\tilde{D}$ residuals are not too small — if $m(\mathbf{X})$ predicts D almost perfectly, you have a weak "instrument" problem.

Diagnostics to Run

Before trusting your estimate:

1. **Check residual $\tilde{D}$** : plot against $\mathbf{X}$. Should look like noise. If systematic patterns remain, your $\hat{m}$ is not fitting well.
2. **Check residual $\tilde{Y}$** : same logic.
3. **Vary the ML method**: if $\hat{\tau}$ changes a lot across choices, your nuisance models are not converging fast enough.
4. **Check overlap**: plot $\hat{m}(\mathbf{X}_i)$ — extreme propensity scores indicate thin overlap regions.

Robustness Workflow

- A. Parametric GLM AME - Conventional applied baseline.
- B. Parametric DML - Same controls, but DML/residualized response-scale estimand.
- C. Flexible outcome DML - ML for $E[Y|X]$, parametric learner for $E[D|X]$.
- D. Flexible treatment DML - Parametric learner for $E[Y|X]$, ML for $E[D|X]$.
- E. Fully flexible DML - ML for both nuisance functions.

Robustness Workflow

- A. Parametric GLM AME - Conventional applied baseline.
- B. Parametric DML - Same controls, but DML/residualized response-scale estimand.
- C. Flexible outcome DML - ML for $E[Y|X]$, parametric learner for $E[D|X]$.
- D. Flexible treatment DML - Parametric learner for $E[Y|X]$, ML for $E[D|X]$.
- E. Fully flexible DML - ML for both nuisance functions.

Comparisons

- A vs B tells how the DML partial linear model machinery changes the estimated treatment effect.
- B vs D, E, F give a sense of how (and if so, where) flexible functional forms make a difference.

Implementation in R: DML Models

```
library(DoubleML)      # or hdm, or build it manually
library(mlr3)
library(mlr3extralearners)

data_ml <- DoubleMLData$new(SLID, y_col = "log_wages", d_cols = "male",
                             x_cols = c("education", "age", "language"))

pr <- lrn("regr.lm")
pc <- lrn("classif.logistic")
npr <- lrn("regr.ranger")
npc <- lrn("classif.ranger")

mod_a <- lm(log_wages ~ sex + age + education + language, data=SLID)
mod_b <- DoubleMLPLR$new(data_ml, ml_l = pr, ml_m = pc, n_folds=5)
mod_b$fit(store_predictions = TRUE)

mod_c <- DoubleMLPLR$new(data_ml, ml_l = npr, ml_m = pc, n_folds=5)
mod_c$fit(store_predictions = TRUE)
mod_d <- DoubleMLPLR$new(data_ml, ml_l = pr, ml_m = npc, n_folds=5)
mod_d$fit(store_predictions = TRUE)
mod_e <- DoubleMLPLR$new(data_ml, ml_l = npr, ml_m = npc, n_folds=5)
mod_e$fit(store_predictions = TRUE)
```



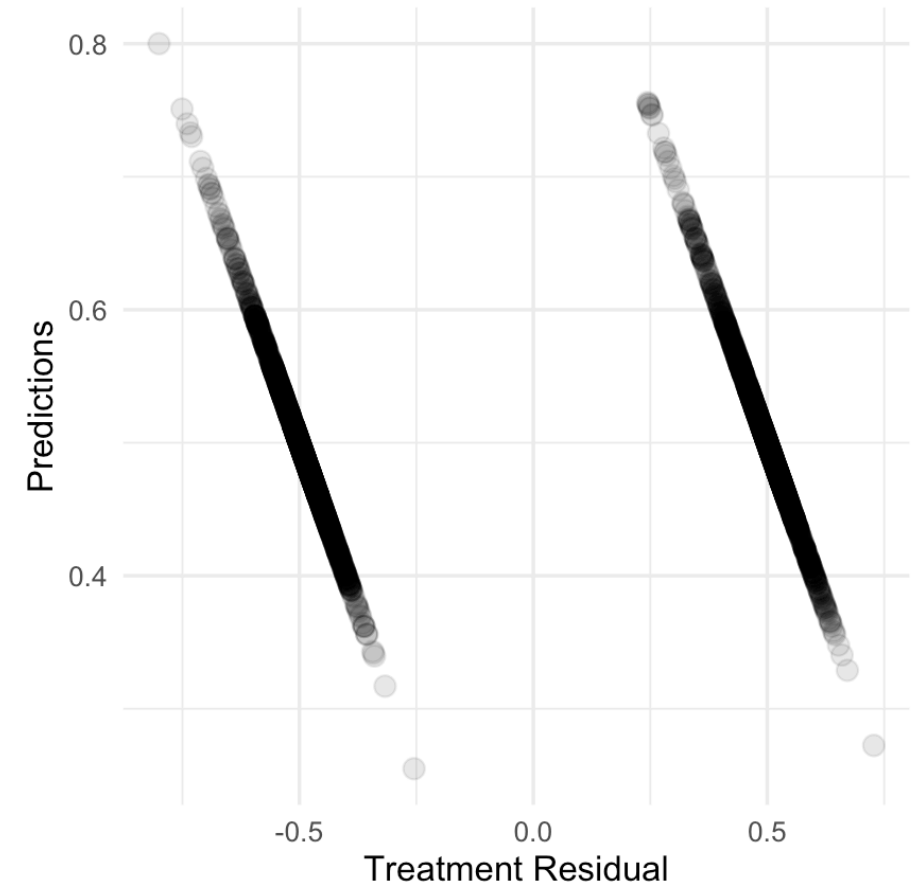
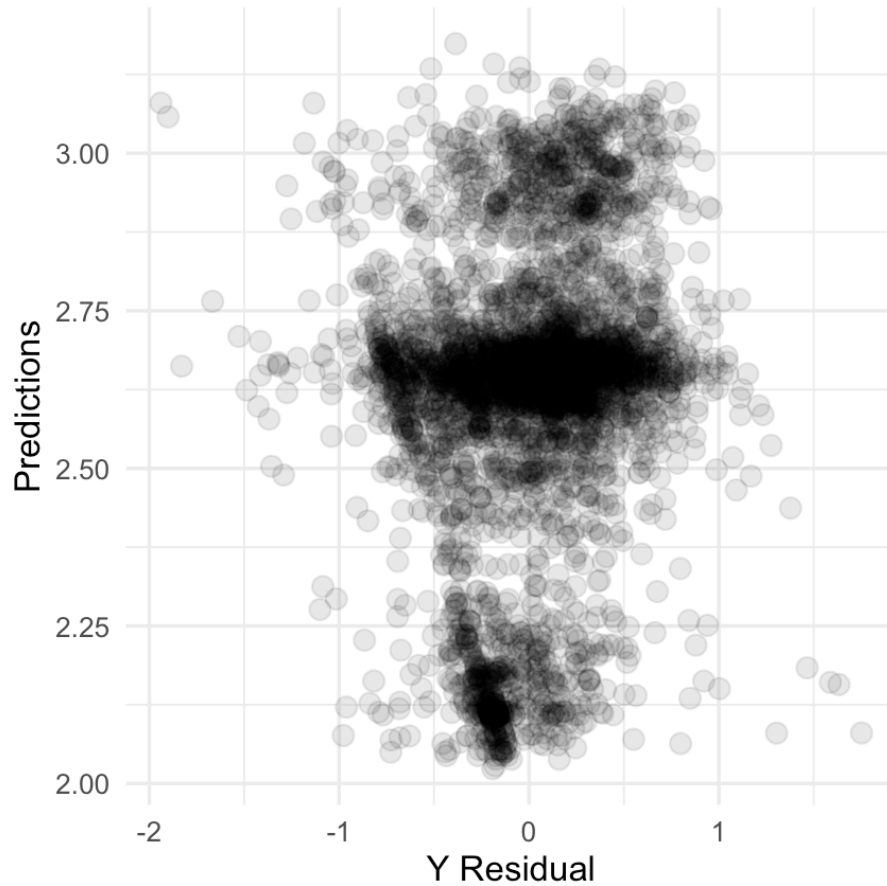
Robustness in R

```
dml_rob <- tibble::tribble(
  ~model, ~coef, ~se,
  "B", mod_b$coef, mod_b$se,
  "C", mod_c$coef, mod_c$se,
  "D", mod_d$coef, mod_d$se,
  "E", mod_e$coef, mod_e$se) |>
mutate(A_est = coef(mod_a)[2],
       A_se = sqrt(diag(vcov(mod_a)))[2])
dml_rob$ovl <- robustness(dnorm, dnorm,
  target_args = list(mean = dml_rob$A_est[1],
                     sd = dml_rob$A_se[1]),
  alt_args = list(mean = dml_rob$coef,
                 sd = dml_rob$se))$ovl
```

dml_rob

```
## # A tibble: 4 × 6
##   model  coef      se A_est  A_se  ovl
##   <chr> <dbl>   <dbl> <dbl> <dbl> <dbl>
## 1 B      0.224 0.0133 0.224 0.0133 0.999
## 2 C      0.215 0.0126 0.224 0.0133 0.706
## 3 D      0.217 0.0132 0.224 0.0133 0.786
## 4 E      0.216 0.0126 0.224 0.0133 0.756
```

Diagnostics in R.





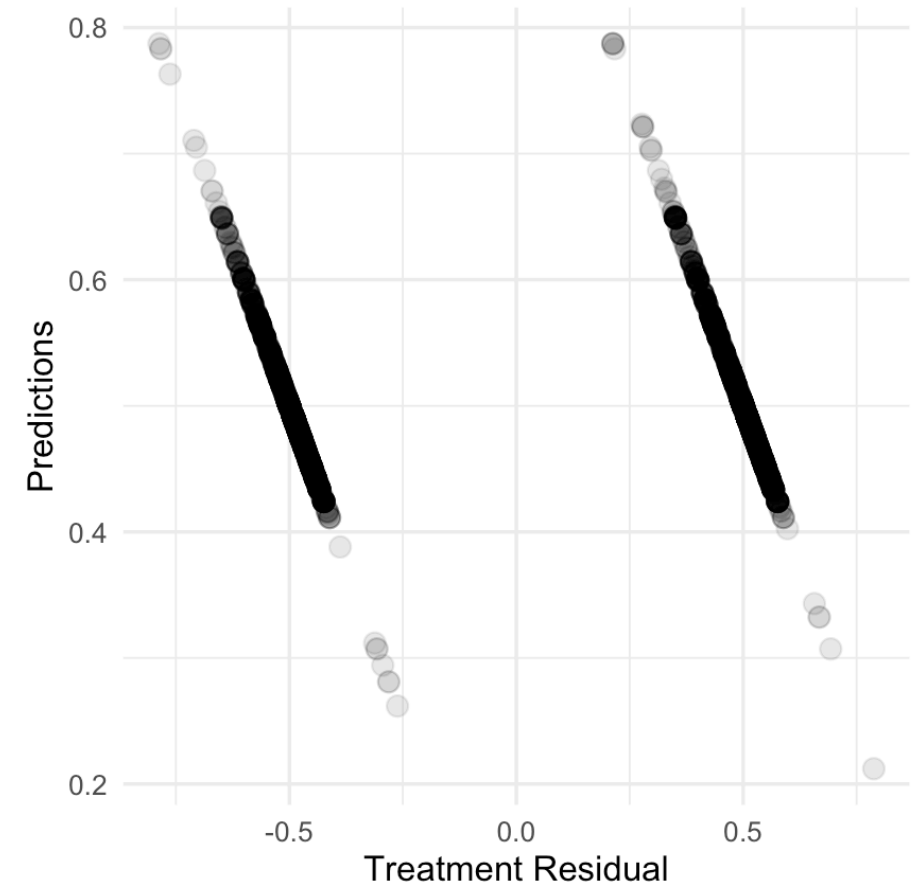
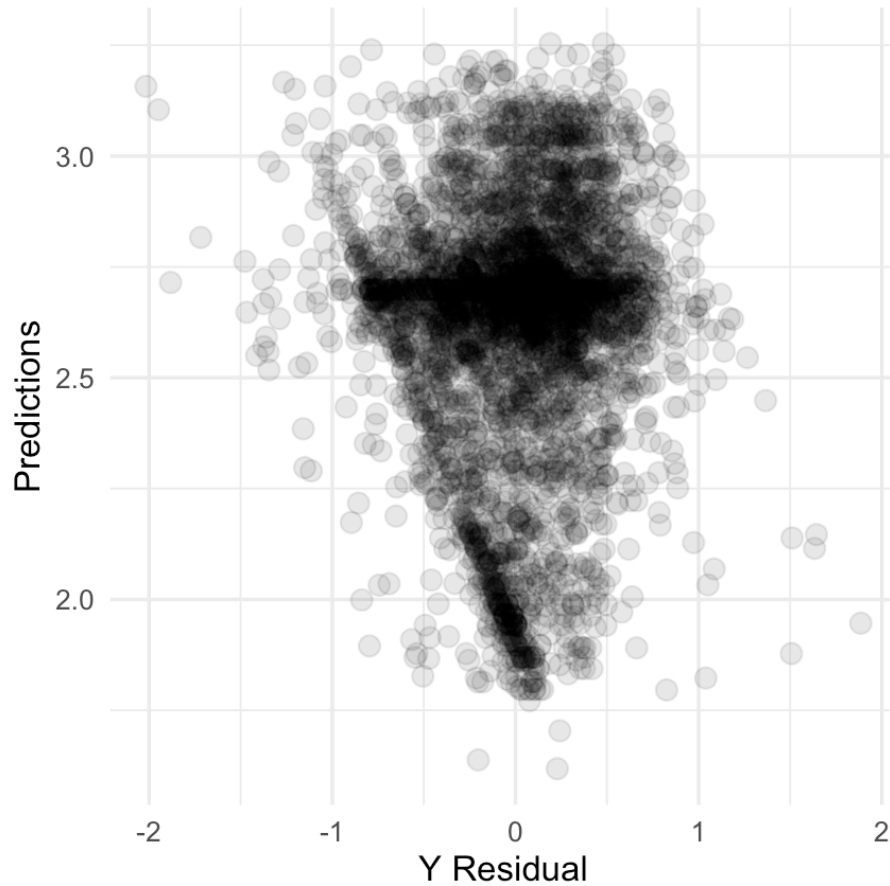
With MARS instead

```
pr <- lrn("regr.lm")
pc <- lrn("classif.logistic")
npr <- lrn("regr.earth")
npc <- lrn("classif.earth")

mod_a <- lm(log_wages ~ sex + age + education + language, data=SLID)
mod_b <- DoubleMLPLR$new(data_ml, ml_l = pr, ml_m = pc, n_folds=5)
mod_b$fit(store_predictions = TRUE)

mod_c <- DoubleMLPLR$new(data_ml, ml_l = npr, ml_m = pc, n_folds=5)
mod_c$fit(store_predictions = TRUE)
mod_d <- DoubleMLPLR$new(data_ml, ml_l = pr, ml_m = npc, n_folds=5)
mod_d$fit(store_predictions = TRUE)
mod_e <- DoubleMLPLR$new(data_ml, ml_l = npr, ml_m = npc, n_folds=5)
mod_e$fit(store_predictions = TRUE)
```

MARS Diagnostics



Robustness

```
dml_rob_mars <- tibble::tribble(
  ~model, ~mars_coef, ~mars_se,
  "B", mod_b$coef, mod_b$se,
  "C", mod_c$coef, mod_c$se,
  "D", mod_d$coef, mod_d$se,
  "E", mod_e$coef, mod_e$se) |>
mutate(A_est = coef(mod_a)[2],
       A_se = sqrt(diag(vcov(mod_a)))[2])
dml_rob_mars$mars_ovl <- robustness(dnorm, dnorm,
  target_args = list(mean = dml_rob_mars$A_est[1],
                     sd = dml_rob_mars$A_se[1]),
  alt_args = list(mean = dml_rob_mars$mars_coef,
                  sd = dml_rob_mars$mars_se))$ovl
```

```
## # A tibble: 4 × 6
##   model A_est rf_coef mars_coef rf_ovl mars_ovl
##   <chr> <dbl>   <dbl>     <dbl> <dbl>   <dbl>
## 1 B      0.224   0.224     0.225  0.999   0.989
## 2 C      0.224   0.215     0.218  0.706   0.793
## 3 D      0.224   0.217     0.223  0.786   0.967
## 4 E      0.224   0.216     0.222  0.756   0.912
```




Robustness Workflow

1. Parametric DML
2. Flexible outcome learner
3. Flexible treatment learner
4. Flexible learners for both
5. Compare treatment effects
6. Compute overlap statistics